

LoreMaster-AI 项目技术方案

基于知识图谱的原神世界观智能问答系统

目录

- 项目背景与动机
- 项目目标
- 数据源分析
- 系统架构设计
- 数据层设计
- 智能Agent设计
- Demo方案设计
- 成本分析
- 项目规划
- 预期成果

第一章 项目背景与动机

1.1 问题陈述

原神(Genshin Impact)作为一款开放世界游戏,构建了一个庞大而复杂的虚拟世界观。这个世界观包含:

- 22,000+ Wiki页面的背景故事
- 170,000+ 条角色对话
- 7个国家的历史与文化
- 数百个角色及其复杂关系网络
- 跨越数千年的时间线

玩家在探索游戏剧情时面临的核心挑战:

信息碎片化: 世界观信息散落在任务对话、书籍、物品描述、角色语音等各处,难以形成完整认知。

关系复杂性: 角色之间存在创造、继承、对抗、联盟等多重关系,传统搜索难以呈现关系网络。

时间跨度大: 故事涉及500年前的灾变、各国历史、当前剧情等多个时间层,因果链难以追溯。

推理需求高: 许多剧情需要玩家自行推理,如"散兵和纳西妲有什么联系"需要多跳关系推理。

1.2 现有方案的局限

传统**Wiki**搜索: 只能返回单一页面, 无法展示实体间关系, 不支持推理类问题。

通用**LLM**: 存在知识截止日期限制, 容易产生幻觉, 无法保证游戏世界观的准确性。

简单**RAG**系统: 仅依赖向量检索, 难以处理需要图结构推理的复杂问题。

1.3 解决方案概述

LoreMaster-AI采用GraphRAG(图增强检索生成)架构, 结合知识图谱的结构化推理能力与大语言模型的自然语言理解能力, 构建一个能够:

- 准确回答事实性问题
- 展示实体间复杂关系
- 进行多跳逻辑推理
- 验证和纠正用户的错误认知
- 理解时间线和因果链

的智能问答系统。

1.4 理论基础

本项目基于以下认知科学理论:

信息觅食理论(**Information Foraging Theory**): 用户在信息环境中的行为类似于动物觅食, 系统应降低用户获取有价值信息的成本。

元认知支架理论(**Metacognitive Scaffolding**): 通过外部工具支持用户的认知过程, 帮助用户建立正确的知识结构。

第二章 项目目标

2.1 学习目标

作为IS596云数据工程课程项目, 本项目旨在实践以下技能:

AWS云服务实践

- S3数据湖架构设计与实现
- Lambda无服务器计算
- Step Functions workflow编排
- EventBridge事件驱动架构
- DynamoDB NoSQL数据库设计
- Glue ETL作业(可选)
- ECS Fargate容器化任务

数据工程实践

- 多源异构数据采集
- ETL流水线设计
- 数据质量管理
- 增量更新策略

AI工程实践

- 知识图谱构建
- 向量嵌入与检索
- LLM应用开发
- Prompt工程

2.2 产品目标

功能目标

- 支持自然语言查询游戏世界观
- 准确回答事实类、关系类、推理类问题
- 能够验证用户声明并纠正错误
- 提供信息来源引用

性能目标

- 响应时间P95 ≤ 8 秒
- 事实查询准确率 $\geq 90\%$
- 关系识别准确率 $\geq 85\%$
- 无幻觉率 $\geq 95\%$

2.3 范围定义

包含范围

- 原神Wiki文本数据处理
- 结构化API数据整合
- 知识图谱构建与查询
- 向量检索系统
- 智能问答Agent
- Web演示界面

不包含范围

- 图像/视频内容处理
- 实时游戏数据同步
- 多语言支持(仅中英文)
- 移动端应用

第三章 数据源分析

3.1 数据源总览

本项目采用三层数据源架构, 确保数据的完整性、准确性和时效性。

第一层: 预构建数据集

主要来源为HuggingFace上的Fandom Wiki多模态数据集:

- 数据规模: 22,162个Wiki页面
- 媒体文件: 184,773个(图片、音频、视频)
- 总大小: 约90GB(文本部分约2-3GB)
- 数据格式: JSONL
- 许可协议: CC-BY-SA 3.0
- 数据截止: 2025年1月17日

GenshinDialog对话语料库:

- 角色数量: 2,422个
- 对话条数: 170,202条
- 游戏版本: 4.5(需注意时效性)

第二层: 结构化API

genshin-db:

- 内容: 角色、武器、圣遗物、材料等游戏实体的结构化数据
- 语言: 支持15种语言
- 更新: 游戏版本更新后1-2天
- 协议: MIT

Project Amber(gi.yatta.moe):

- 内容: 同上, 另有Python封装库(ambr-py)
- 更新: 同日更新
- 协议: GPL v3

第三层: 原始游戏数据

GenshinData(GitHub):

- ExcelBinOutput: 游戏配置数据
- TextMap: 多语言文本映射
- Readable: 游戏内书籍和文档
- Subtitle: 对话字幕

3.2 数据特点分析

数据优势

- 覆盖全面: 从官方Wiki到原始游戏文件, 信息完整

- 多源互补: API提供结构化属性, Wiki提供叙事内容
- 持续更新: 跟随游戏版本迭代

数据挑战

- 格式异构: Markdown、JSON、JSONL等多种格式
- 内容重叠: 同一信息在多个数据源中重复出现
- 质量参差: Wiki内容为社区编辑, 存在不一致
- 实体指代: 同一角色可能有多个名称(如"纳西妲"/"小吉祥草王"/"草神")

3.3 数据更新策略

数据源	更新频率	触发方式	更新类型
Wiki数据集	每6周	游戏版本更新后	全量替换
genshin-db	每日	定时任务	增量更新
Project Amber	每6小时	定时任务	增量更新
GenshinData	实时/每小时	Webhook/定时	增量更新

第四章 系统架构设计

4.1 整体架构

系统采用分层架构设计, 包含以下核心层次:

数据采集层

- 负责从外部数据源获取原始数据
- 支持全量采集和增量更新
- 使用EventBridge进行调度触发
- 数据存储至S3原始数据区

数据处理层

- 基于Step Functions编排的ETL流水线
- 包含解析、筛选、清洗、切片、嵌入、抽取等阶段
- 使用Lambda进行轻量处理, Glue处理大规模数据
- 输出存储至S3处理数据区

数据服务层

- Neo4j AuraDB: 知识图谱存储与查询
- Pinecone: 向量索引与语义检索
- DynamoDB: 元数据和缓存存储

应用服务层

- 智能Agent: 意图识别、查询路由、推理验证
- API服务: 对外提供查询接口
- Web前端: Streamlit演示界面

4.2 技术选型

计算服务

服务	用途	选择理由
AWS Lambda	ETL处理、API服务	按需付费、无需管理服务器
AWS Glue	大规模ETL	托管Spark、适合大数据处理
ECS Fargate	大文件下载	无超时限制、适合长时任务

存储服务

服务	用途	选择理由
Amazon S3	数据湖存储	低成本、高可靠、支持生命周期
DynamoDB	元数据存储	免费层充足、毫秒级延迟
Neo4j AuraDB	图数据库	免费层可用、Cypher查询强大
Pinecone	向量数据库	免费层可用、检索性能优秀

AI服务

服务	用途	选择理由
Amazon Bedrock	LLM调用	AWS原生集成、多模型可选
Claude API	复杂推理	推理能力强、支持长上下文
Titan Embeddings	向量化	成本低、质量可靠

编排服务

服务	用途	选择理由
Step Functions	流水线编排	可视化、错误处理完善

EventBridge 事件调度 灵活触发、几乎免费

4.3 数据流转全景

数据在系统中的完整流转路径：

阶段一：数据采集 外部数据源 → 采集服务 (ECS/Lambda) → S3原始数据区

阶段二：数据解析 S3原始数据 → Parse Lambda → 标准化文档 (Parquet格式)

阶段三：数据筛选 (**Demo**阶段) 标准化文档 → Filter Lambda → 须弥子集数据

阶段四：数据清洗 筛选后数据 → Clean Lambda → 去重、实体消歧、标准化

阶段五：并行处理

- 向量分支：清洗数据 → Chunk → Embed → 向量数据
- 图谱分支：清洗数据 → NER → Extract → 三元组数据

阶段六：数据加载

- 向量数据 → Pinecone索引
- 三元组数据 → Neo4j图谱
- 元数据 → DynamoDB表

阶段七：应用服务 用户查询 → Agent处理 → 检索/推理 → 响应生成

第五章 数据层设计

5.1 S3数据湖结构

数据湖采用分区存储策略，按数据处理阶段和来源组织：

原始数据区 (**raw/**)

- wiki/{date}/: Wiki数据集，按采集日期分区
- api/{source}/{date}/: API数据，按来源和日期分区
- github/{commit}/: GitHub数据，按提交哈希分区

处理数据区 (**processed/**)

- documents/: 清洗后的标准化文档
- chunks/: 文本切片
- embeddings/: 向量数据 (临时，加载后可删除)
- triples/: 三元组数据 (临时，加载后可删除)

元数据区 (**metadata/**)

- schemas/: 数据Schema定义
- manifests/: 处理清单
- quality_reports/: 质量报告

归档区 (archive/)

- 历史数据归档, 使用Glacier存储类

5.2 ETL处理流水线

流水线由Step Functions编排, 包含以下阶段:

Parse阶段

- 输入: 原始JSONL/JSON文件
- 处理: 格式转换、Schema验证、字段映射
- 输出: 标准化Parquet文件
- 耗时: 约5分钟

Filter阶段 (Demo特有)

- 输入: 解析后的全量文档
- 处理: 基于配置的关键词和分类筛选
- 输出: 须弥子集文档
- 耗时: 约2分钟
- 说明: 全量运行时跳过此阶段

Clean阶段

- 输入: 筛选后文档
- 处理: 去重(内容哈希)、实体消歧、别名标准化
- 输出: 清洗后文档
- 耗时: 约10分钟

Chunk阶段

- 输入: 清洗后文档
- 处理: 递归文本分割, 保留元数据
- 配置: 块大小512 tokens, 重叠50 tokens
- 输出: 文本块列表
- 耗时: 约5分钟

Embed阶段

- 输入: 文本块
- 处理: 调用Bedrock生成1536维向量
- 输出: 向量数据
- 耗时: 约20-30分钟(受API限制)

Extract阶段

- 输入:清洗后文档
- 处理:NER识别 + LLM三元组抽取
- 输出:实体列表和关系三元组
- 耗时:约30-60分钟(受API限制)

Load阶段

- Pinecone加载:批量upsert向量
- Neo4j加载:执行Cypher创建节点和关系
- DynamoDB加载:写入元数据记录
- 耗时:约15-30分钟

5.3 知识图谱Schema设计

节点类型

类型	说明	示例
Character	角色	纳西妲、散兵、艾尔海森
Location	地点	须弥、净善宫、教令院
Organization	组织	须弥教令院、愚人众、七神
Event	事件	坎瑞亚灾变、虚空神计划
Item	物品	草神之心、知识之冕
Concept	概念	禁忌知识、世界树

关系类型

关系	说明	示例
CREATED	创造	大慈树王 → 纳西妲
BORN_FROM	诞生于	纳西妲 → 世界树
SUCCEDED	继任	纳西妲 → 大慈树王
MEMBER_OF	成员	阿赫玛尔 → 教令院
LOCATED_IN	位于	净善宫 → 须弥城
CONFINED	囚禁	教令院 → 纳西妲
CONSPIRED_AGAINST	密谋反对	阿赫玛尔 → 纳西妲

CAUSED	导致	灾变 → 禁忌知识扩散
DESTROYED	摧毁	灾变 → 坎瑞亚
ALLIED_WITH	结盟	旅行者 → 纳西妲

5.4 向量索引设计

索引配置

- 维度: 1536
- 相似度度量: 余弦相似度
- 索引算法: HNSW

元数据字段

- chunk_id: 文本块唯一标识
- entity_id: 关联实体ID
- entity_type: 实体类型
- section: 所属章节
- source_url: 来源URL
- content: 原始文本(用于展示)

5.5 DynamoDB表设计

entity_metadata表

- 分区键: entity_type
- 排序键: entity_id
- 属性: 名称、别名、属性、来源URL、关联ID

data_versions表

- 分区键: source_id
- 排序键: resource_id
- 属性: 内容哈希、更新时间、状态、统计信息

alias_mapping表

- 分区键: alias
- 属性: 规范ID、规范名称、实体类型

5.6 增量更新机制

变更检测

1. 获取最新数据
2. 计算内容哈希(SHA256)
3. 与DynamoDB中的历史哈希对比

4. 识别新增、修改、删除的记录

增量处理

- 仅处理发生变更的实体
- 重新生成相关chunks和embeddings
- 重新抽取相关三元组
- 使用upsert/merge更新数据库

一致性保障

- 批量写入使用事务
- 支持失败重试
- 保留版本历史支持回滚

第六章 智能Agent设计

6.1 Agent架构总览

Agent采用模块化设计, 包含以下核心组件:

意图识别模块:理解用户查询的目的 问题分解模块:将复杂问题拆分为子问题 检索路由模块:决定使用图谱查询还是向量检索 图谱查询引擎:执行Cypher查询获取结构化信息 向量检索引擎:执行语义检索获取相关文本 逻辑验证模块:验证用户声明的正确性 响应生成模块:整合信息生成自然语言回答

6.2 意图识别

意图分类体系

意图类型	说明	示例
fact_query	事实查询	"纳西妲的元素是什么?"
relationship_query	关系查询	"纳西妲和大慈树王是什么关系?"
multi_hop_reasoning	多跳推理	"散兵和纳西妲有什么联系?"
fact_verification	事实验证	"散兵是雷电将军的弟弟吗?"
timeline_query	时间线查询	"500年前发生了什么?"
exploration	开放探索	"给我讲讲纳西妲的故事"
comparison	比较分析	"纳西妲和雷神有什么不同?"

causal_analysis 因果分析 "为什么纳西妲会被囚禁？"

三层识别架构

- 第一层:规则匹配(高置信度、低成本)
- 第二层:分类器(中等场景、中等成本)
- 第三层:LLM推理(复杂场景、高成本)

6.3 检索路由策略

不同意图类型对应不同的检索策略：

意图	主要检索方式	辅助检索方式
fact_query	图谱直接查询	-
relationship_query	图谱路径查询	向量补充背景
multi_hop_reasoning	图谱多跳查询	向量补充上下文
fact_verification	图谱查询 + NLI验证	-
timeline_query	图谱时间过滤	向量补充叙事
exploration	向量语义检索	图谱补充关系

6.4 图谱查询引擎

查询类型

简单属性查询：

- 直接根据实体ID获取属性
- 使用模板化Cypher

关系查询：

- 查询两实体间的直接关系
- 返回关系类型和属性

路径查询：

- 查找两实体间的最短路径
- 支持指定最大跳数

聚合查询：

- 统计某类实体数量

- 查找符合条件的实体集合

实体消歧

- 用户输入 → 别名表匹配 → 规范实体ID
- 支持模糊匹配和多语言别名

6.5 向量检索引擎

查询增强策略

同义词扩展：

- 将"草神"扩展为"草神 OR 纳西妲 OR Nahida"

上下文增强：

- 基于对话历史补充查询上下文

HyDE(假设文档嵌入)：

- 先让LLM生成假设答案, 再用答案检索

混合检索

- 语义检索: 基于向量相似度
- 关键词检索: 基于BM25
- 结果融合: 使用RRF(倒数排名融合)

6.6 逻辑验证模块

这是实现"认知支架"的核心模块, 用于验证用户声明。

验证流程

1. 用户声明: "散兵是雷电将军的弟弟"
2. 三元组提取: (散兵, SIBLING_OF, 雷电将军)
3. 图谱查询: 查找散兵和雷电将军的实际关系
4. 关系比对: 实际关系为 CREATED(创造), 非 SIBLING_OF
5. 语义验证: 使用NLI模型确认语义差异
6. 生成纠正: "散兵不是雷电将军的弟弟, 而是她创造的人偶"

验证结果类型

-  一致(Consistent): 用户说法正确
-  矛盾(Contradiction): 用户说法错误
-  部分正确(Partial): 部分正确但不精确
-  未知(Unknown): 知识库中无相关信息

关系语义映射

- 用户口语表达 → 图谱标准关系
- "弟弟/哥哥" → SIBLING_OF
- "造的/制造的" → CREATED
- "属于/加入" → MEMBER_OF
- "继承/继任" → SUCCEEDED

6.7 响应生成

生成策略

信息整合：

- 去重: 移除重复信息
- 排序: 按相关性和重要性排序
- 筛选: 保留与问题最相关的内容

结构规划：

- 简单问题: 直接回答
- 复杂问题: 分段组织, 逻辑清晰

来源引用：

- 标注信息来源URL
- 区分确定信息和推测信息

后续建议：

- 根据当前话题推荐相关问题
- 引导用户深入探索

第七章 Demo方案设计

7.1 Demo目标

技术验证

- 端到端流水线跑通
- 各组件正常协作
- 性能满足基本要求

能力验证

- Agent意图识别准确
- 图谱查询正确
- 多跳推理有效
- 事实验证可靠

可扩展性验证

- Demo与全量使用相同架构
- 移除筛选即可扩展到全量

7.2 数据范围选择

选择须弥(Sumeru)作为Demo数据集：

选择理由

- 故事线完整且相对独立
- 涉及核心世界观(七神、世界树、禁忌知识)
- 角色关系复杂(创造、继承、对抗、阴谋)
- 时间跨度大(500年前至今)
- 有明确因果链可测试推理

数据范围

- 角色:约30个(须弥角色 + 关联角色)
- 地点:约15个
- 组织:约8个
- 事件:约10个
- 物品:约6个
- 概念:约6个
- 总计:约70个实体, 200-500个关系

7.3 筛选策略

与全量方案的一致性

- 使用完全相同的采集流程
- 使用完全相同的ETL流水线
- 仅在Parse后增加Filter阶段
- Filter通过配置文件控制, 可一键关闭

筛选条件

基于Wiki Categories:

- "Sumeru", "Sumeru Characters", "Sumeru Locations"
- "Akademiya", "Dendro Characters", "Archons"

基于实体名称:

- 须弥角色白名单
- 关联角色白名单(散兵、旅行者等)
- 须弥地点关键词

基于内容关键词:

- "须弥"、"Sumeru"、"教令院"、"Akademiya"
- "纳西妲"、"Nahida"、"世界树"、"Irminsul"

7.4 核心故事线

Demo涵盖的主要故事线：

500年前：灾变时期

- 坎瑞亚灾变爆发
- 禁忌知识从坎瑞亚扩散
- 禁忌知识侵蚀世界树
- 大慈树王决定牺牲自己
- 大慈树王创造纳西妲
- 大慈树王从世界树中抹除自己
- 所有人遗忘大慈树王的存在
- （同期稻妻）雷电将军创造散兵

500年间：沉寂时期

- 纳西妲被教令院囚禁于净善宫
- 散兵流浪后加入愚人众
- 愚人众改造散兵，植入雷神之心

现在：游戏剧情

- 阿赫玛尔与愚人众密谋
- 计划用散兵制造"虚空神"取代纳西妲
- 旅行者抵达须弥，结识纳西妲
- 虚空神计划启动
- 纳西妲和旅行者进入世界树
- 发现大慈树王的真相
- 散兵背叛愚人众
- 纳西妲净化世界树
- 纳西妲被承认为真正的草神

7.5 测试用例设计

测试类型与数量

类型	数量	测试重点
事实查询	15	属性准确性
关系查询	12	关系识别
多跳推理	10	路径推理

事实验证	8	纠错能力
时间线	5	时序理解
开放探索	5	信息整合
总计	55	-

示例测试用例

事实查询：

- "纳西妲的元素是什么？" → 草 (Dendro)
- "散兵是几星角色？" → 5星
- "净善宫位于哪里？" → 须弥城

关系查询：

- "纳西妲和大慈树王是什么关系？" → 创造者与被创造者、继任关系
- "教令院和纳西妲是什么关系？" → 囚禁关系

多跳推理：

- "散兵和纳西妲有什么联系？" → 通过阿赫玛尔的虚空神计划关联
- "坎瑞亚灾变如何影响了纳西妲的诞生？" → 完整因果链

事实验证：

- "散兵是雷电将军的弟弟吗？" → 错误，是她创造的人偶
- "纳西妲是大慈树王的女儿吗？" → 不精确，是她创造的

7.6 验收标准

指标	目标值
流水线成功率	100%
数据完整性	≥95%
事实查询准确率	≥90%
关系查询准确率	≥85%
多跳推理成功率	≥75%
事实纠错准确率	≥80%
响应时间P95	≤8秒

无幻觉率

≥95%

第八章 成本分析

8.1 Demo阶段成本

一次性处理成本

项目	用量	成本
ECS Fargate	0.5 vCPU × 1小时	\$0.04
Lambda ETL	约200次调用	\$2.00
S3存储与请求	200MB + 10K请求	\$0.10
Bedrock Embedding	约1M tokens	\$0.10
Bedrock Claude	约2M tokens	\$6.00
Step Functions	约500状态转换	\$0.01
小计	-	约\$8-10

月度运维成本

项目	成本
S3存储	\$0.01
Lambda查询服务	\$0.10
Neo4j AuraDB Free	\$0
Pinecone Starter	\$0
小计	约\$0.15

开发测试期间

项目	估算
多次重跑Pipeline	\$40-50
Agent调试	\$30-40

测试运行	\$5-10
小计	约\$75-100

Demo全周期总成本:\$85-115

在AWS Educate \$100额度范围内可完成。

8.2 全量方案成本

Lambda为主方案(推荐)

月度成本约\$25-35:

- AWS服务(ETL+存储): \$8-10
- LLM API: \$17-25
- Neo4j AuraDB Free: \$0
- Pinecone Starter: \$0

完全AWS原生方案

月度成本约\$115-165:

- Neptune: \$50-80
- OpenSearch: \$30-50
- 其他服务: \$35

不推荐用于学生项目, 超出Educate额度。

8.3 成本优化策略

免费服务利用

- Neo4j AuraDB Free: 200K节点免费
- Pinecone Starter: 100K向量免费
- DynamoDB: 25GB免费存储
- Lambda: 每月100万次免费调用

按需付费优化

- 使用Lambda替代持续运行的服务
- S3生命周期策略自动归档
- 合理设置Lambda内存和超时

第九章 项目规划

9.1 开发路线图

第一周:基础设施与配置

- AWS环境搭建(S3、IAM、DynamoDB)
- 外部服务配置(Neo4j、Pinecone)
- 筛选配置编写

第二周:数据采集与ETL开发

- 数据采集Lambda开发
- Parse与Filter Lambda开发
- Clean与Chunk Lambda开发

第三周:LLM集成与数据加载

- Bedrock集成(Embed、Extract)
- 数据加载Lambda开发
- Step Functions编排

第四周:Agent开发

- 意图识别模块
- 图谱查询与向量检索模块
- 推理与验证模块

第五周:测试与优化

- 测试用例编写与执行
- Bug修复与性能优化
- 测试报告生成

第六周:前端与文档

- Streamlit演示界面
- 技术文档编写
- 项目汇报准备

9.2 里程碑

里程碑	时间	交付物
M1	第2周末	ETL流水线可运行
M2	第3周末	数据成功加载至数据库
M3	第4周末	Agent核心功能完成
M4	第5周末	通过验收测试

9.3 风险与应对

风险	可能性	影响	应对措施
LLM抽取质量不稳定	中	高	优化Prompt、人工审核关键数据
AWS Educate额度不足	低	高	严格成本监控、优先使用免费服务
图谱查询性能不足	中	中	优化Cypher、添加索引
数据源API变更	低	中	监控API状态、准备备用数据源

第十章 预期成果

10.1 技术成果

完整的云数据工程实践

- 掌握AWS S3数据湖设计
- 掌握Lambda无服务器架构
- 掌握Step Functions workflow编排
- 掌握EventBridge事件驱动
- 掌握DynamoDB NoSQL设计
- 实践Terraform基础设施即代码

GraphRAG系统实现

- 知识图谱构建与查询
- 向量嵌入与检索
- 混合检索策略
- LLM应用集成

10.2 产品成果

智能问答系统

- 支持事实查询、关系查询、多跳推理、事实验证
- 准确率满足验收标准
- 响应时间可接受

Web演示界面

- 聊天交互界面
- 知识图谱可视化

- 查询过程展示

10.3 文档成果

- 系统架构设计文档
- 数据流转说明文档
- API接口文档
- 用户使用指南
- 项目总结报告

10.4 扩展可能

数据扩展

- 从须弥扩展到全部7个国家
- 增加更多数据源
- 支持实时更新

功能扩展

- 多语言支持
- 图像内容理解
- 对话记忆与个性化

部署扩展

- 迁移到AWS原生服务(Neptune、OpenSearch)
- 添加CDN和负载均衡
- 实现高可用架构

附录A: 核心实体清单 (Demo范围)

角色 (约30个)

- 神明: 纳西妲、大慈树王、雷电将军
- 教令院: 阿赫玛尔、艾尔海森、卡维、提纳里、赛诺
- 愚人众: 散兵、博士
- 其他: 迪希雅、妮露、坎蒂丝、莱依拉、旅行者、派蒙等

地点 (约15个)

- 国家: 须弥、坎瑞亚、稻妻
- 须弥内部: 须弥城、净善宫、教令院、桓那兰那、阿如村等

组织 (约8个)

- 须弥教令院、六大学派、愚人众、镀金旅团、七神等

事件(约10个)

- 坎瑞亚灾变、禁忌知识侵蚀、大慈树王牺牲、纳西妲诞生
- 纳西妲囚禁、虚空神计划、散兵叛变、世界树净化等

物品(约6个)

- 草神之心、雷神之心、知识之冕、虚空神躯壳等

概念(约6个)

- 禁忌知识、世界树、神之心、天理等

附录B: 关系类型清单

关系	方向	示例
CREATED	创造者→被造物	大慈树王→纳西妲
BORN_FROM	被造物→来源	纳西妲→世界树
SUCCEEDED	继任者→前任	纳西妲→大慈树王
MEMBER_OF	成员→组织	阿赫玛尔→教令院
LEADER_OF	领导者→被领导	纳西妲→须弥
LOCATED_IN	小地点→大地点	净善宫→须弥城
CONFINED	囚禁者→被囚者	教令院→纳西妲
CONFINED_IN	被囚者→地点	纳西妲→净善宫
CONSPIRED_AGAINST	密谋者→目标	阿赫玛尔→纳西妲
CAUSED	原因→结果	灾变→禁忌知识扩散
DESTROYED	破坏者→被破坏	灾变→坎瑞亚
ERASED_FROM	被抹除者→来源	大慈树王→世界树
CORRUPTED	侵蚀者→被侵蚀	禁忌知识→世界树
MODIFIED	改造者→被改造	愚人众→散兵
POSSESSES	拥有者→物品	纳西妲→草神之心

ALLIED_WITH	盟友↔盟友	旅行者↔纳西妲
BETRAYED	背叛者→被背叛	散兵→愚人众

附录C: 测试用例分类

事实查询(15例)

- 属性查询: 元素、武器、稀有度等
- 状态查询: 角色当前状态
- 归属查询: 组织成员关系
- 位置查询: 地点层级关系

关系查询(12例)

- 直接关系: 两实体间的直接联系
- 双向关系: 互为盟友、朋友等
- 多重关系: 同时存在多种关系

多跳推理(10例)

- 路径发现: 寻找间接联系
- 因果链: 事件序列推理
- 跨域关联: 跨国家、跨时间的联系

事实验证(8例)

- 关系类型纠错: 弟弟→人偶
- 状态纠错: 还活着→已消失
- 事实纠错: 支持→囚禁

时间线(5例)

- 时间点查询: 某时发生的事件
- 事件排序: 前后顺序判断
- 时间跨度: 持续时间查询

开放探索(5例)

- 角色介绍: 完整叙述角色故事
- 组织介绍: 组织背景和成员
- 概念解释: 世界观概念阐述

文档结束

以上就是完整的项目方案文档内容，你可以直接复制保存为PDF。